# Manual de despliegue OpenVPN en AWS con Terraform desde Ubuntu y GitHub

*Flujo recomendado: Ubuntu -> GitHub -> Terraform -> AWS -> OpenVPN*

**Objetivo:** Conectar un servidor o equipo Ubuntu con GitHub, clonar el repositorio privado de OpenVPN, instalar las herramientas necesarias, configurar las variables del cliente, desplegar la infraestructura con Terraform y crear perfiles OpenVPN para usuarios.

## Flujo general

- 1. Preparar Ubuntu y conectarlo con GitHub.
- 2. Clonar el repositorio terraform-aws-openvpn.
- 3. Instalar Terraform y AWS CLI.
- 4. Configurar credenciales AWS.
- 5. Crear y ajustar terraform.tfvars.
- 6. Validar y desplegar con Terraform.
- 7. Validar el servidor OpenVPN.
- 8. Crear y descargar perfiles .ovpn.
- 9. Actualizar el repositorio cuando se hagan cambios.

# 1. Prerrequisitos

- Equipo o servidor Ubuntu con acceso a Internet.
- Cuenta de GitHub con acceso al repositorio cloudtingsas/terraform-aws-openvpn.
- Cuenta AWS con permisos para crear EC2, Security Groups, Elastic IP y Key Pair.
- VPC existente, subnet pública existente y CIDR de la VPC.
- Usuario con permisos sudo en Ubuntu.

**Seguridad:** No almacene Access Key, Secret Key, archivos .pem, perfiles .ovpn, terraform.tfvars ni estados de Terraform dentro del repositorio Git.

# 2. Preparar Ubuntu y conectar con GitHub

## 2.1 Actualizar el sistema e instalar Git

```
sudo apt update
sudo apt install -y git openssh-client
git --version
ssh -V
```

## 2.2 Configurar identidad de Git

```
git config --global user.name "Nombre Apellido"
git config --global user.email "correo@empresa.com"
git config --global --list
```

## 2.3 Crear una llave SSH para GitHub

```
ssh-keygen -t ed25519 -C "correo@empresa.com"
```

Presione Enter para usar la ruta sugerida. Se recomienda proteger la llave privada con una passphrase cuando el equipo sea de uso administrativo.

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub
```

Copie el contenido mostrado y agréguelo en GitHub en Settings > SSH and GPG keys > New SSH key.

## 2.4 Probar la conexión con GitHub

```
ssh -T git@github.com
```

La primera vez puede solicitar confirmar la huella del host. Si la autenticación es correcta, GitHub confirmará que la llave fue aceptada.

**Alternativa:** También puede clonar por HTTPS, pero para repositorios privados GitHub requiere autenticación con un token o un método autorizado. Para servidores Ubuntu administrativos, SSH suele ser más cómodo para operaciones repetidas.

## 3. Clonar el repositorio OpenVPN

Cree una carpeta de trabajo y clone el repositorio:

```
mkdir -p ~/proyectos
cd ~/proyectos
git clone git@github.com:cloudtingsas/terraform-aws-openvpn.git
cd terraform-aws-openvpn
git status
ls -la
```

La estructura esperada debe incluir, como mínimo:

```
main.tf
variables.tf
outputs.tf
terraform.tfvars.example
README.md
scripts/crear-cliente-openvpn.sh
docs/
```

**Importante:** Si el script crear-cliente-openvpn.sh todavía no está integrado dentro del aprovisionamiento de Terraform, el manual incluye más adelante cómo copiarlo manualmente a la instancia EC2.

## 4. Instalar Terraform en Ubuntu

Instale Terraform desde el repositorio oficial de HashiCorp:

```
sudo apt-get update && sudo apt-get install -y gnupg software-properties-common curl
curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(. /etc/os-release && echo $VERSION_CODENAME) main" |
sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update
sudo apt install -y terraform
terraform version
```

## 5. Instalar AWS CLI

Para Ubuntu x86_64:

```
sudo apt update
sudo apt install -y unzip curl
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
aws --version
```

Si el servidor Ubuntu es ARM64, debe utilizar el instalador correspondiente a aarch64 en lugar de x86_64.

## 6. Configurar credenciales AWS

No escriba credenciales dentro de main.tf. Configure AWS CLI:

```
aws configure --profile openvpn
```

Ingrese AWS Access Key ID, AWS Secret Access Key, región y formato de salida. Luego active el perfil:

```
export AWS_PROFILE=openvpn
aws sts get-caller-identity
```

**Validación:** No continúe con Terraform hasta que aws sts get-caller-identity muestre correctamente la cuenta y el principal IAM con el que se realizará el despliegue.

## 7. Configurar el proyecto para el cliente

### 7.1 Crear terraform.tfvars

```
cp terraform.tfvars.example terraform.tfvars
nano terraform.tfvars
```

Ejemplo de variables, manteniendo la estructura del proyecto actual:

```
aws_region = "us-east-1"
vpc_id = "vpc-xxxxxxxxxxxxxxxxx"
vpc_cidr = "10.60.0.0/16"
subnet_id = "subnet-xxxxxxxxxxxxxxxxx"
name_prefix = "BastionOpenVPN"
key_name = "BastionOpenVPN"
instance_type = "t3.micro"
openvpn_allowed_cidrs = ["203.0.113.10/32"]
initial_client_name = "cliente1"
create_eip = true
```

## 7.2 Significado de las variables

| Variable | Uso |
|---|---|
| aws_region | Región AWS donde se desplegará OpenVPN. |
| vpc_id | VPC donde se creará la instancia EC2. |
| vpc_cidr | Red de la VPC que será accesible a través de la VPN. |
| subnet_id | Subnet pública donde se ubicará el servidor OpenVPN. |
| name_prefix | Prefijo para los recursos creados. |
| key_name | Nombre de la llave SSH generada para administrar la instancia. |
| instance_type | Tipo de instancia EC2. |
| openvpn_allowed_cidrs | CIDR públicos autorizados para conectarse al puerto OpenVPN. |
| initial_client_name | Nombre del primer perfil OpenVPN. |
| create_eip | Define si se asigna Elastic IP al servidor. |

# 8. Desplegar OpenVPN con Terraform

Desde la raíz del repositorio:

```
cd ~/proyectos/terraform-aws-openvpn
```

## 8.1 Inicializar

```
terraform init
```

## 8.2 Formatear y validar

```
terraform fmt -recursive
terraform validate
```

## 8.3 Revisar el plan

```
terraform plan
```

Revise especialmente región, VPC, subnet, tipo de instancia, Security Group, Elastic IP y nombres de recursos.

## 8.4 Aplicar

```
terraform apply
```

Cuando Terraform solicite confirmación, escriba yes. Al finalizar, consulte los outputs:

```
terraform output
terraform output ssh_command
terraform output download_client_ovpn_command
```

# 9. Acceder a la instancia OpenVPN

Si Terraform generó localmente la llave .pem, restrinja sus permisos:

```
chmod 400 BastionOpenVPN.pem
```

Obtenga el comando SSH desde Terraform o utilice la IP pública:

```
ssh -i BastionOpenVPN.pem ubuntu@IP_PUBLICA
```

# 10. Validar que OpenVPN está funcionando

```
sudo systemctl status openvpn-server@server
sudo ss -lunp | grep 1194
sudo cat /etc/openvpn/server/server.conf
ls -lh ~/client-configs/files/
```

**Resultado esperado:** El servicio debe estar activo, el servidor debe escuchar en UDP 1194 y debe existir al menos el perfil inicial configurado durante el despliegue.

# 11. Script para crear usuarios OpenVPN

## 11.1 Si Terraform ya instala el script

Dentro de la instancia EC2:

```
ls -l ~/crear-cliente-openvpn.sh
~/crear-cliente-openvpn.sh nombre_usuario
~/crear-cliente-openvpn.sh carlos.rojas
```

El perfil debe quedar en una ruta similar a:

```
/home/ubuntu/client-configs/files/carlos.rojas.ovpn
```

## 11.2 Si el script solo está en GitHub y todavía no se instala automáticamente

Desde el Ubuntu donde clonó el repositorio, copie el script a la instancia EC2:

```
scp -i BastionOpenVPN.pem scripts/crear-cliente-openvpn.sh
ubuntu@IP_PUBLICA:/home/ubuntu/crear-cliente-openvpn.sh
```

Luego ingrese por SSH y habilite ejecución:

```
ssh -i BastionOpenVPN.pem ubuntu@IP_PUBLICA
chmod +x ~/crear-cliente-openvpn.sh
~/crear-cliente-openvpn.sh cliente2
```

# 12. Descargar el archivo .ovpn

Desde el equipo administrador:

```
scp -i BastionOpenVPN.pem ubuntu@IP_PUBLICA:~/client-configs/files/carlos.rojas.ovpn .
```

El archivo .ovpn se importa posteriormente en OpenVPN Connect en el equipo del usuario.

**Seguridad:** Cada usuario debe tener un perfil .ovpn independiente. No publique archivos .ovpn ni llaves .pem en GitHub, correo abierto, chats públicos o repositorios compartidos.

## 13. Actualizar el repositorio en Ubuntu

Cuando el repositorio cambie en GitHub, actualice la copia local antes de trabajar:

```
cd ~/proyectos/terraform-aws-openvpn
git status
git pull origin main
```

Si usted modifica documentación, scripts o Terraform desde Ubuntu:

```
git status
git add .
git commit -m "chore: actualizar plantilla OpenVPN"
git pull origin main --rebase
git push origin main
```

**Antes del push:** Revise siempre git status y confirme que no aparezcan terraform.tfvars, .pem, .ovpn, .tfstate o credenciales.

## 14. .gitignore recomendado

```
.terraform/
*.tfstate
*.tfstate.*
terraform.tfvars
*.auto.tfvars
*.auto.tfvars.json
*.pem
*.key
*.ovpn
.env
.env.*
.DS_Store
```

## 15. Buenas prácticas de seguridad

- Restringir SSH/22 únicamente a IPs administrativas o a una red de gestión.
- Evitar 0.0.0.0/0 en openvpn_allowed_cidrs salvo que exista una justificación operativa. Preferir /32 de las IP públicas autorizadas cuando sean estables.
- Permitir únicamente UDP 1194 para OpenVPN, salvo que la configuración del proyecto utilice otro puerto/protocolo.
- No reutilizar el mismo .ovpn entre varios usuarios.
- Mantener inventario de usuarios creados y revocar perfiles cuando un usuario deje de requerir acceso.
- Guardar el estado Terraform en un backend seguro si el proyecto pasa de una administración individual a un proceso de equipo/producción.
- Revisar terraform plan antes de cada terraform apply.

## 16. Eliminación de la infraestructura

Si se requiere eliminar los recursos creados por esta plantilla:

```
terraform destroy
```

Revise el plan de destrucción y confirme escribiendo yes solo cuando esté seguro.

## 17. Checklist de despliegue

- Ubuntu actualizado y Git instalado.
- Llave SSH registrada en GitHub y ssh -T git@github.com validado.
- Repositorio clonado correctamente.
- Terraform y AWS CLI instalados.
- aws sts get-caller-identity validado.
- terraform.tfvars creado con datos correctos del cliente.
- terraform validate sin errores.
- terraform plan revisado.
- terraform apply completado.
- Servicio openvpn-server@server activo.
- UDP 1194 escuchando.
- Perfil inicial .ovpn generado y probado.
- Proceso de creación de usuarios validado.
- Archivos sensibles fuera de GitHub.